Experiment No.: 07
Title: Implement the Q-Learning Algorithm2

Batch: B3 **Roll No.: 16014225801** **Experiment No.: 07**

Aim: To study, design, and implement the model-free Q-Learning algorithm to solve discrete state-action decision problems through dynamic optimal control and reward maximization.

Resources needed: Any programming language.

Pre Requisite: Machine learning concepts, Statistics

Theory:

1. Introduction to Reinforcement Learning (RL)

Reinforcement Learning is a paradigm where an agent learns to make optimal decisions by interacting with an environment. The agent receives feedback in the form of numerical rewards or penalties, aiming to maximize cumulative long-term reward.

2. Markov Decision Process (MDP)

The core mathematical framework for RL is defined as an MDP with a 4-tuple (S, A, P, R):

- **S**: Finite set of environment states.
- **A**: Finite set of allowed actions.
- **P(s'| s, a)**: State transition probability function.
- **R(s, a)**: Scalar reward function returned after executing action a in state s.

3. Q-Learning Algorithm

Q-Learning is an off-policy, model-free Temporal Difference (TD) control algorithm. It maintains a **Q-Table** of dimensions \vert{}S\vert{} \times \vert{}A\vert{}, where each entry Q(s, a) represents the expected cumulative reward of taking action a in state s and thereafter following the optimal policy.

4. Bellman Optimality Equation & Update Rule

The core value update for state s and action a upon observing next state s' and reward r is given by:

Q(s, a) -> Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]

Where:

- \alpha \in (0, 1] is the **Learning Rate** (determines how strongly new information overwrites old data).
- \gamma \in [0, 1) is the **Discount Factor** (balances immediate vs. future rewards).
- \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] is the **Temporal Difference (TD) Error**.

5. Exploration vs. Exploitation (\epsilon-Greedy Policy)

To balance discovering new transitions and exploiting current optimal paths, an \epsilon-greedy policy is used:

- With probability \epsilon, select a random action (Exploration).
- With probability 1 - \epsilon, select action \arg\max_a Q(s, a) (Exploitation).

- Decay \epsilon over successive episodes to converge toward an optimal deterministic policy.

Procedure:

1.**Launch Environment & Install Dependencies:**

Initialize the Python virtual environment and install required dependencies (numpy, gymnasium, matplotlib).

Bash

pip install numpy gymnasium matplotlib

Initialize Environment & Q-Table:

Set up a discrete grid-world environment (e.g., FrozenLake-v1 or a custom matrix grid). Create a Q-table initialized to zeros with size (\text{State Space Size} \times \text{Action Space Size}).

Configure Hyperparameters:

Define parameters: Learning Rate \alpha = 0.1, Discount Factor \gamma = 0.99, initial Exploration \epsilon = 1.0, Exploration Decay Rate = 0.001, and Total Episodes = 1000.

Implement Learning Loop:

- For each episode, reset the environment to state s.
- Choose action a using the \epsilon-greedy rule.
- Step the environment to obtain s', reward r, and termination status (done).
- Update entry Q(s, a) using the Bellman update formula.
- Transition to state s = s' until episode completion.

Decay \epsilon & Evaluate Convergence:

Decay \epsilon after each episode. Plot the total cumulative rewards per episode to verify policy convergence.

Results: (Steps with screenshots)

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# ------------------------------------------------------------
# 1. LOAD DATASET
# ------------------------------------------------------------

file_path = "india_2000_2024_daily_weather.csv"

df = pd.read_csv(file_path, parse_dates=["date"])

# Sort data chronologically for each city
df = df.sort_values(["city", "date"]).reset_index(drop=True)
```

```
print("Dataset loaded successfully.")
print("Dataset shape:", df.shape)
print("Number of cities:", df["city"].nunique())
print("Cities:", list(df["city"].unique()))
print("Date range:",
      df["date"].min().date(),
      "to",
      df["date"].max().date())


# ------------------------------------------------------------
# 2. CREATE DISCRETE WEATHER STATES
# ------------------------------------------------------------

# Temperature:
# 0 = Low
# 1 = Moderate
# 2 = High
temp_bins = [-np.inf, 20, 30, np.inf]

# Rainfall:
# 0 = Dry
# 1 = Light Rain
# 2 = Heavy Rain
rain_bins = [-np.inf, 0.1, 10, np.inf]

# Wind:
# 0 = Low
# 1 = Moderate
# 2 = High
wind_bins = [-np.inf, 15, 30, np.inf]

df["temp_state"] = pd.cut(
    df["temperature_2m_max"],
    bins=temp_bins,
    labels=False
)

df["rain_state"] = pd.cut(
    df["precipitation_sum"],
    bins=rain_bins,
    labels=False
)

df["wind_state"] = pd.cut(
    df["wind_speed_10m_max"],
    bins=wind_bins,
    labels=False
)
```

```
# Combine 3 variables into one discrete state.
#
# Temperature has 3 values
# Rain has 3 values
# Wind has 3 values
#
# Total states = 3 x 3 x 3 = 27

df["state"] = (
    df["temp_state"] * 9
    + df["rain_state"] * 3
    + df["wind_state"]
)


# ------------------------------------------------------------
# 3. GET NEXT DAY RAINFALL
# ------------------------------------------------------------

# The agent makes a decision based on today's weather.
# Reward is calculated using the next day's rainfall.

df["next_rain"] = (
    df.groupby("city")["rain_sum"]
    .shift(-1)
)

# Handle final observation of each city
df["next_rain"] = df["next_rain"].fillna(0)


# ------------------------------------------------------------
# 4. DEFINE ENVIRONMENT
# ------------------------------------------------------------

cities = df["city"].unique()

# Store each city's chronological data separately
city_data = {
    city: data.reset_index(drop=True)
    for city, data in df.groupby("city")
}


# ------------------------------------------------------------
# 5. Q-LEARNING PARAMETERS
# ------------------------------------------------------------

n_states = 27
n_actions = 3

# Q-table
```

```
Q = np.zeros((n_states, n_actions))

# Hyperparameters
alpha = 0.1
gamma = 0.99

epsilon = 1.0
epsilon_decay = 0.001
min_epsilon = 0.01

episodes = 1000
episode_length = 30

# Random generator
rng = np.random.default_rng(42)


# ------------------------------------------------------------
# 6. REWARD FUNCTION
# ------------------------------------------------------------

def calculate_reward(action, rainfall):

    rainy = rainfall > 0.1

    # Action 0: No preparation
    if action == 0:

        if rainy:
            return -1.0
        else:
            return 0.2

    # Action 1: Carry umbrella
    elif action == 1:

        if rainy:
            return 1.0
        else:
            return -0.2

    # Action 2: Avoid outdoor activity
    elif action == 2:

        if rainy:
            return 0.7
        else:
            return -0.5


# ------------------------------------------------------------
```

```
# 7. EPSILON-GREEDY POLICY
# ------------------------------------------------------------

def choose_action(state, epsilon):

    # Exploration
    if rng.random() < epsilon:
        return rng.integers(n_actions)

    # Exploitation
    return np.argmax(Q[state])


# ------------------------------------------------------------
# 8. Q-LEARNING TRAINING
# ------------------------------------------------------------

episode_rewards = []

print("\nStarting Q-Learning training...")

for episode in range(episodes):

    # Select random city
    city = rng.choice(cities)

    data = city_data[city]

    # Select random starting point
    start = rng.integers(
        0,
        len(data) - episode_length - 1
    )

    total_reward = 0

    # Run one episode
    for step in range(episode_length):

        current_row = data.iloc[start + step]

        # Current state
        state = int(current_row["state"])

        # Select action using epsilon-greedy policy
        action = choose_action(state, epsilon)

        # Next state
        next_row = data.iloc[start + step + 1]

        next_state = int(next_row["state"])
```

```
        # Reward based on next day's rainfall
        reward = calculate_reward(
            action,
            current_row["next_rain"]
        )

        # ---------------------------------------------------
        # Q-Learning Bellman Update
        # ---------------------------------------------------

        old_q_value = Q[state, action]

        best_next_q = np.max(Q[next_state])

        new_q_value = (
            old_q_value
            + alpha
            * (
                reward
                + gamma * best_next_q
                - old_q_value
            )
        )

        Q[state, action] = new_q_value

        total_reward += reward

    episode_rewards.append(total_reward)

    # ---------------------------------------------------
    # Epsilon decay
    # ---------------------------------------------------

    epsilon = max(
        min_epsilon,
        epsilon * (1 - epsilon_decay)
    )


# ---------------------------------------------------------
# 9. DISPLAY TRAINING RESULTS
# ---------------------------------------------------------

print("\nTraining completed.")

print("Number of states:", n_states)
print("Number of actions:", n_actions)
print("Learning rate:", alpha)
print("Discount factor:", gamma)
```

```
print("Initial epsilon:", 1.0)
print("Final epsilon:", round(epsilon, 4))
print("Total episodes:", episodes)

print(
    "Average reward - first 100 episodes:",
    round(np.mean(episode_rewards[:100]), 3)
)

print(
    "Average reward - last 100 episodes:",
    round(np.mean(episode_rewards[-100:]), 3)
)


# ------------------------------------------------------------
# 10. GREEDY POLICY EVALUATION
# ------------------------------------------------------------

test_rewards = []

action_counts = np.zeros(
    n_actions,
    dtype=int
)

for _ in range(100):

    city = rng.choice(cities)

    data = city_data[city]

    start = rng.integers(
        0,
        len(data) - episode_length - 1
    )

    total_reward = 0

    for step in range(episode_length):

        row = data.iloc[start + step]

        state = int(row["state"])

        # Greedy action
        action = np.argmax(Q[state])

        action_counts[action] += 1

        reward = calculate_reward(
```

```
            action,
            row["next_rain"]
        )

        total_reward += reward

    test_rewards.append(total_reward)


print("\nEvaluation Results")

print(
    "Average evaluation reward:",
    round(np.mean(test_rewards), 3)
)

print(
    "Total evaluation reward:",
    round(np.sum(test_rewards), 3)
)

print(
    "Action counts:",
    action_counts.tolist()
)


# ------------------------------------------------------------
# 11. DISPLAY LEARNED POLICY
# ------------------------------------------------------------

policy = np.argmax(Q, axis=1)

print("\nLearned Policy")

for state in range(n_states):

    print(
        "State",
        state,
        "-> Action",
        policy[state],
        "Q-values:",
        np.round(Q[state], 3)
    )


# ------------------------------------------------------------
# 12. PLOT CUMULATIVE REWARD
# ------------------------------------------------------------

plt.figure(figsize=(10, 5))
```

```
plt.plot(
    episode_rewards,
    alpha=0.5,
    label="Episode Reward"
)

# Moving average
window = 50

moving_average = (
    pd.Series(episode_rewards)
    .rolling(window)
    .mean()
)

plt.plot(
    moving_average,
    linewidth=2,
    label="50-Episode Moving Average"
)

plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Q-Learning Training Performance")
plt.legend()
plt.grid(True)

plt.show()


# ------------------------------------------------------------
# 13. DISPLAY Q-TABLE
# ------------------------------------------------------------

q_table = pd.DataFrame(
    Q,
    columns=[
        "No Preparation",
        "Carry Umbrella",
        "Avoid Outdoor"
    ]
)

print("\nFinal Q-Table:")
print(q_table.round(3))
```

Output:

```
Dataset loaded successfully.
Dataset shape: (91320, 12)
Number of cities: 10
Cities: ['Ahmedabad', 'Bangalore', 'Chennai', 'Delhi',
         'Hyderabad', 'Jaipur', 'Kolkata', 'Lucknow',
         'Mumbai', 'Pune']

Date range: 2000-01-01 to 2024-12-31

Starting Q-Learning training...

Training completed.

Number of states: 27
Number of actions: 3
Learning rate: 0.1
Discount factor: 0.99
Initial epsilon: 1.0
Final epsilon: 0.3677
Total episodes: 1000

Average reward - first 100 episodes: 0.609
Average reward - last 100 episodes: 6.232

Evaluation Results

Average evaluation reward: 7.252
Total evaluation reward: 725.2
Action counts: [778, 1842, 380]
```

Questions:

1. What is the fundamental difference between model-free (Q-Learning) and model-based reinforcement learning?

Model-free reinforcement learning learns how to make decisions directly from interactions with the environment without explicitly learning the transition probabilities or reward model of the environment. Q-Learning is a model-free algorithm because it directly learns the Q-values of state-action pairs using observed rewards and subsequent states. In contrast, model-based reinforcement learning builds or uses an explicit model of the environment, including how the environment changes from one state to another and what rewards are expected. The model can then be used to simulate future outcomes and plan actions. Therefore, the fundamental difference is that Q-Learning learns the optimal action values directly from experience, whereas model-based reinforcement learning uses an environment model for planning and decision-making.

2. Explain the purpose of the exploration factor (ε) and why ε-decay is crucial during training.

The exploration factor ε controls the balance between exploration and exploitation during reinforcement learning. During exploration, the agent randomly selects actions to discover potentially better state-action combinations, while during exploitation it selects the action having the highest current Q-value. A high ε at the beginning of training is useful because the agent has little knowledge about the environment and needs to explore different actions. As training progresses, ε is gradually reduced through ε-decay so that the agent increasingly relies on the knowledge stored in the Q-table. This is important because continuously exploring random actions can prevent the agent from following the learned policy effectively, while reducing ε too quickly can cause the agent to miss potentially better actions. In this experiment, ε decreases from 1.0 to approximately 0.3677 after 1000 episodes, allowing substantial exploration initially while gradually increasing exploitation of learned Q-values.

Conclusion

The model-free Q-Learning algorithm was successfully implemented using the India daily weather dataset from 2000 to 2024. The continuous weather measurements were converted into 27 discrete states using temperature, precipitation, and wind-speed categories, while three possible weather-related decisions were defined as actions. A Q-table was initialized and updated using the Bellman optimality equation with a learning rate of 0.1 and discount factor of 0.99. Using an ε-greedy strategy, the agent initially explored different actions and gradually learned a policy through ε-decay. The average reward increased from 0.609 in the first 100 episodes to 6.232 in the last 100 episodes, while greedy evaluation achieved an average reward of 7.252. Thus, the experiment demonstrates how Q-Learning can learn an optimal decision policy from historical discrete state-action experiences without explicitly constructing a model of the environment.

Grade: AA / AB / BB / BC / CC / CD /DD

Signature of faculty in-charge with date

References:

Websites:

1. *Reinforcement Learning: An Introduction* — Richard S. Sutton and Andrew G. Barto, MIT Press.
2. *Gymnasium Documentation* — gymnasium.farama.org